Comparative Evaluation of Language Models for Code Generation on Benchmark Datasets

by

Pulla Sandeep

Sandeepa Sarvothama Prabhu

Ananthu Nair

Jyothirmayi Chukkapalli

Report

Submitted in partial fulfillment of the requirements

for the DLFA Program

Centre for Continuing Education

Indian Institute of Science

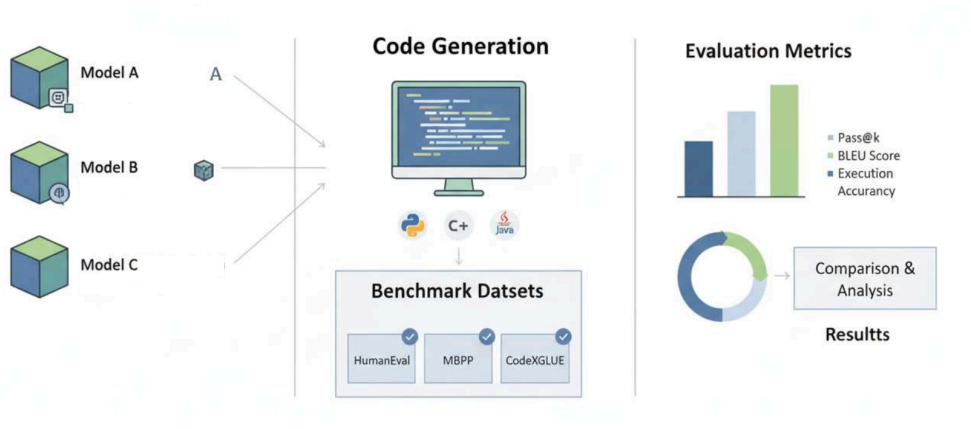
Bangalore – 560 012 India

Abstract

The rapid advancement of Large Language Models (LLMs) has transformed automated code generation from a research curiosity into a practical capability with significant industrial relevance. This work presents a comprehensive evaluation framework for assessing LLM-based code generation systems, focusing on dimensions such as functional correctness, syntactic validity, robustness to ambiguous specifications, and efficiency of developer interaction. We benchmarked state-of-the-art foundation models to evaluate code generation for Python, as well as studied the LLM adaptability for other higher level programming languages like C & Java.

Our findings highlight substantial variance in performance across different foundational models, underscoring the need for fine-tuning, domain adaptation, and better prompting and context engineering. Industry surveys and adoption case studies reveal strong interest in adapting LLM-driven code generation for software development acceleration, test code development and automation, and overall developer productivity. However, organizations consistently report challenges related to model alignment, hallucination control, compliance, and integration with existing engineering workflows.

In this study we evaluated the primary code generation capabilities of the state of the art LLMs like CodeLlama, StarCoder, and Deepseek using the open source datasets like HumanEval and MBPP, and CodeXGLUE. We built end-to-end evaluation metrics for the python code generation including functional correctness (pass@k), AST (Abstract Syntax Tree) analysis, effect of LLM temperature, output performance (Solutions per second) and comparison of performance for each model.



To maintain the uniformity and comparability, each of the model evaluations is conducted on the NVIDIA A100 GPU with 80GB GPU memory in the Google Colab Environment.

Acknowledgments

We extend our sincere gratitude to Professor Dr. Shirish Shevade for guiding us in the capstone project. We also appreciate our Capstone Project mentor Mr. Karthick Raja for his valuable feedback and support during the weekly review sessions.

We sincerely thank the faculty of IISc Bangalore for their dedicated teaching throughout the course. We are also grateful to the mentors for sharing their extensive technical expertise and guidance during this time. Lastly, we extend our heartfelt thanks to the chief mentor Mr. Prakash Mudholkar, program manager Mr. Abdul Azeez and all TalentSprint team members for their unwavering support and readiness to address our concerns and provide guidance throughout the entire program.

Table of Contents

Introduction

Problem Statement

Large Language Models (LLMs) such as Code Llama, StarCoder, DeepSeek Coder have demonstrated significant progress in generating source code directly from natural language instructions.
These models provide developers with powerful tools that can automate programming tasks, ranging from simple functions to moderately complex algorithmic solutions.
It will be a good study to explore some unanswered questions regarding their relative strengths, weaknesses, and failure modes in real-world programming scenarios.
Another issue is the reliability and correctness of AI-generated code. While LLMs frequently generate syntactically correct programs, they often fail on functional correctness, missing edge cases, or producing logically incorrect solutions. Such risks become critical when these models are applied in mission-critical systems like healthcare or finance.
Moreover, LLMs vary significantly in their ability to generalize across multiple programming languages (Python, C, C++, Java) and benefit differently from techniques such as prompt engineering or fine-tuning. Therefore, this project aims to systematically evaluate and compare modern LLMs (Code Llama, StarCoder, DeepSeek Coder) across multiple benchmarks (HumanEval, MBPP etc.), perform detailed error analysis, and investigate the role of prompt design and possible fine-tuning in improving correctness.

Background information:

Domain Information

- **Natural Language Processing (NLP)** : A subfield of Artificial Intelligence (AI) focused on enabling machines to understand, generate, and interact using human language. It forms the foundation for tasks such as translation, summarization, and code generation.
- **Large Language Models (LLMs)** : Deep transformer-based neural architectures trained on massive text corpora to learn syntactic and semantic patterns of language. They exhibit emergent capabilities such as reasoning, in-context learning, and code synthesis when fine-tuned on programming datasets.
- **Code Generation Models** : Specialized LLMs (e.g., Code Llama, StarCoder) trained on large-scale code repositories to translate natural language prompts into executable code. These models leverage multi-language code (Python, C++, Java, etc.) to generalize across programming paradigms.
- **HumanEval Benchmark** : A standardized dataset for evaluating the functional correctness of code generated by LLMs. It consists of Python programming tasks with test cases, allowing objective comparison of model performance based on code accuracy and pass rates.

- **Prompt Engineering**: Techniques to optimize model behavior. Prompt engineering crafts input queries to elicit high-quality code responses, for improved relevance.

Problem Description & Analysis

Modern software development demands rapid, reliable, and context-aware code generation, yet writing syntactically correct and semantically meaningful code remains a highly skilled and time-consuming task. With the emergence of LargeLanguage Models (LLMs) trained on both natural language and source code, it is now possible to automatically generate code snippets, functions, or even entire programs from text based descriptions. However, achieving consistent accuracy, interpretability, and generalization across diverse programming tasks remains a significant challenge.

Existing code generation models such as Code Llama, StarCoder, and Codex demonstrate impressive results but still struggle with logical consistency, handling long dependencies, and understanding ambiguous natural language prompts.

Additionally, evaluating generated code quality requires not only syntactic correctness but also functional accuracy, typically tested using benchmarks like HumanEval. Thus, a robust solution is needed that can evaluate, compare, and enhance modern LLMs for code generation, focusing on accuracy, efficiency, and contextual alignment with natural language requirements.

Problem Analysis

Code generation is inherently complex as it involves mapping unstructured human language to structured, executable programming logic while preserving intent and correctness.

Limitations of current methods:

LLMs sometimes generate syntactically correct but logically incorrect code due to incomplete understanding of problem constraints. Gaps occur when models trained on one code domain (e.g., Python) fail to perform well on others (e.g., C++, Java).

Lack of robust evaluation metrics to measure functional success.

Key challenges

Semantic Understanding: Aligning natural language instructions with programming intent. Functional Evaluation: Ensuring generated code not only compiles but passes all test cases.

Prompt Sensitivity: Reducing the model's dependency on specific phrasing or token order in prompts.

Cross-Language Adaptability: Maintaining performance across multiple programming languages.

Model Efficiency: Balancing model size, inference speed, and output accuracy for practical deployment.

Use cases for code LLMs

AI as an assistant to developers - Enhancing developer productivity and workflow efficiency

1. Code generation (from NL text prompts, flow diagrams, design documents)

2. Code understanding (explain the functionality)
3. Unit test case generation (including edge case identification)
4. Optimizing existing code
5. AI-assisted programming (Code Assistance & Completion Tools)
6. Rapid prototyping of algorithms
7. Bug Detection & Code Refactoring
8. Automated Documentation & Comment Generation
9. Evaluating different implementation approaches

Motivation for the Study

LLMs for code are a cutting-edge research area bridging NLP, software engineering, and AI safety.

- **Industry**: Organizations increasingly rely on AI-assisted coding to reduce development time, improve quality, and bridge skill gaps in programming.
- **Conceptual understanding**: The project explores the application of cutting-edge transformer-based architectures in translating natural language instructions into executable code. Deep dive into the internals of these models.
- Opportunity to contribute by experimenting with multiple open models and publishing insights.
- Potential to develop novel techniques that improve model outputs.
- **Research**: This work combines advancements in NLP, machine reasoning, and software automation. It contributes to the ongoing research in evaluating LLMs for code understanding, code completion, and reasoning over structured programming languages.

Models and Datasets:

Models:

StarCoder:

StarCoder is a family of open-weight large language models (LLMs) optimized for code generation and understanding, created by BigCode, a collaboration between Hugging Face and ServiceNow.

Deepseek Coder:

DeepSeek-Coder-V2, an open-source Mixture-of-Experts (MoE) code language model that achieves performance comparable to GPT4-Turbo in code-specific tasks. Specifically, DeepSeek-Coder-V2 is further pre-trained from an intermediate checkpoint of DeepSeek-V2 with additional 6 trillion tokens. Through this continued pre-training, DeepSeek-Coder-V2 substantially enhances the coding and mathematical reasoning capabilities of DeepSeek-V2, while maintaining comparable performance in general language tasks.

CodeLlama:

CodeLlama is a family of open-source large language models developed by Meta AI for code generation and understanding. It is built on top of the LLaMA 2 architecture, fine-tuned specifically for coding tasks.

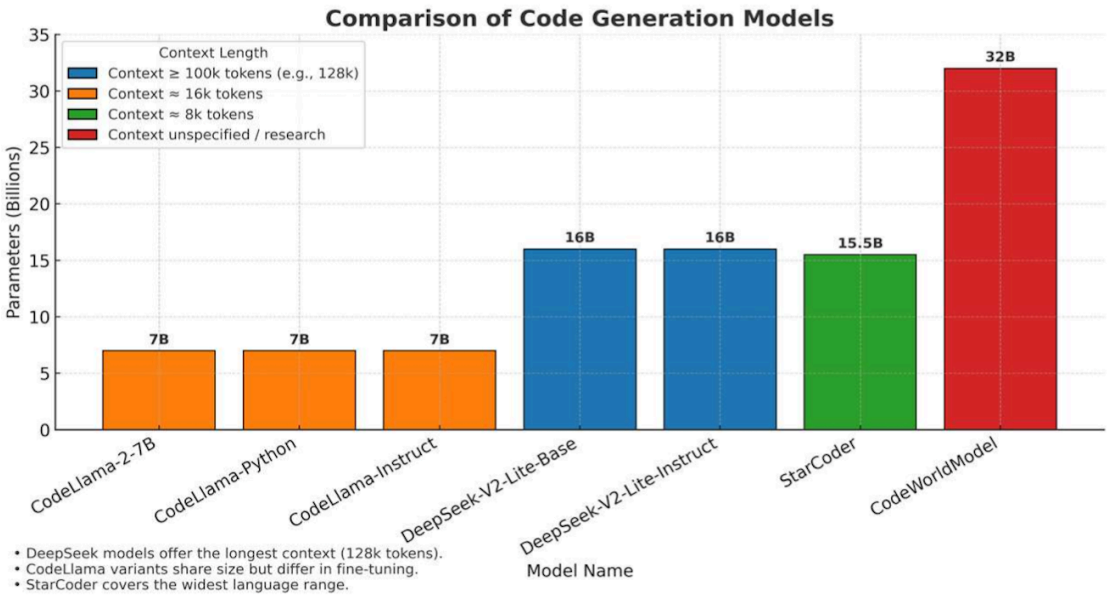
Code World Model (Meta CWM):

Code World Model (CWM) is a 32-billion-parameter open-weights LLM, to advance research on code generation with world models released in September 2025 by Meta. The model has undergone mid-training on a large amount of observation-action trajectories from Python interpreter and agentic Docker environments, and performs extensive multi- task reasoning RL in verifiable coding, math, and multi-turn software engineering environments. CWM is a dense, decoder-only LLM trained with a context size of up to 131 k tokens. Independent of its world modeling capabilities, CWM offers strong performance on general coding and math tasks.

Comparison of code LLMs Used for this Study

Model	Params	Context	Specialty	Notes
CodeLlama-2-7B	7B	16k	Base	General code model
CodeLlama-Python	7B	16k	Python	Python fine-tuned
CodeLlama-Instruct	7B	16k	Instruction	Instruction-following
DeepSeek-V2-Lite-Base	16B (2.4B active)	128k	MoE	Large context, efficient
DeepSeek-V2-Lite-Instruct	16B (2.4B active)	128k	Instruction	Tuned for code tasks
StarCoder	15.5B	8k	Multilingual	80+ languages
CodeWorldModel (Meta)	32B	–	Dense	Research model

Context Length Comparison



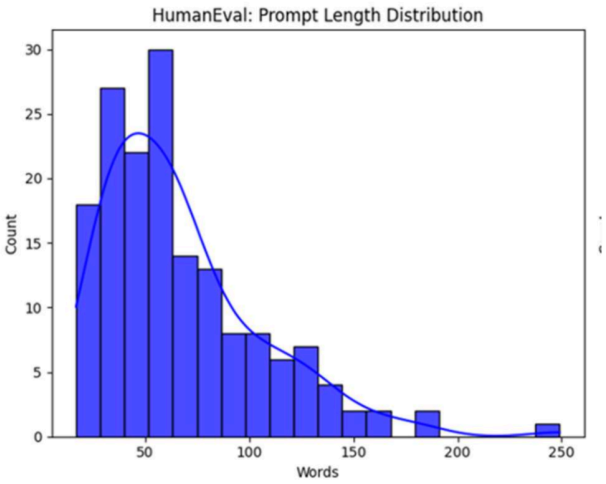
Datasets

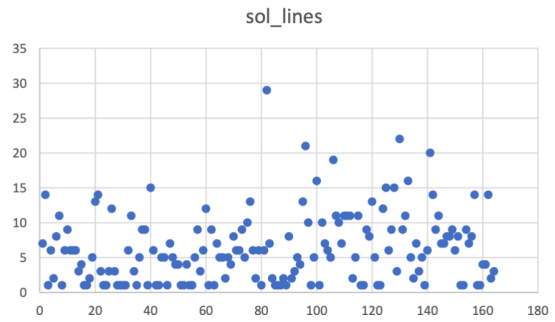
HumanEval

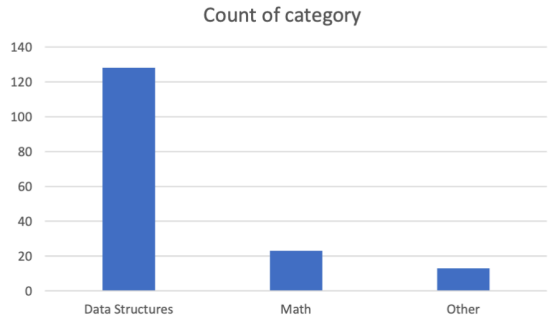
The HumanEval dataset, introduced by OpenAI, contains 164 Python programming problems designed to evaluate the functional correctness of code generated by language models. Each problem includes:

- A function signature defining the task interface.
- A docstring describing the problem in natural language.
- Hidden unit tests to validate generated solutions.
- HumanEval supports the pass@k metric for benchmarking model
- performance based on test case success rates.

Purpose: Provides a standardized evaluation benchmark for measuring the ability of LLMs to synthesize executable and functionally correct code from natural language descriptions.

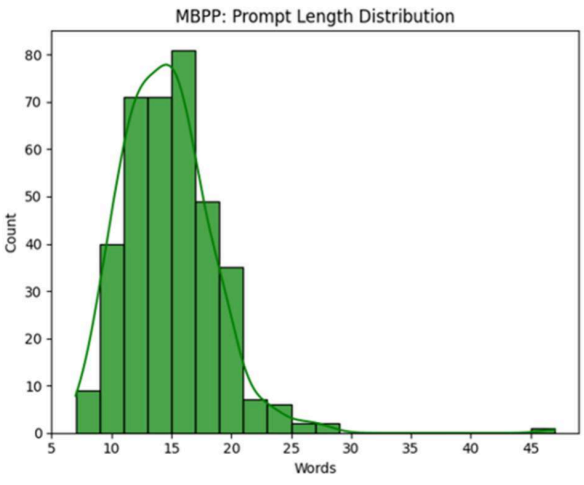


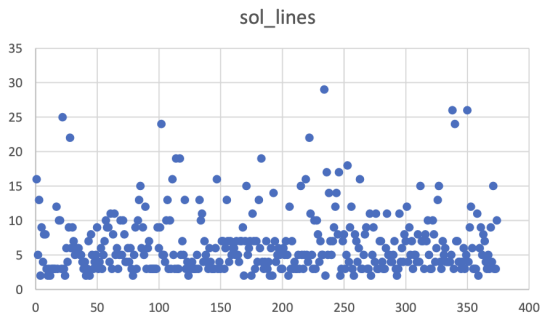


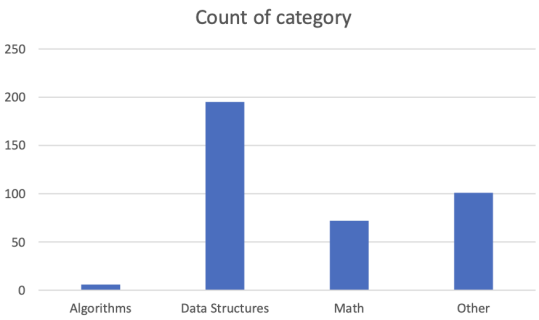


Mostly Basic Python Problems (MBPP)

Dataset (Introduced by Austin et al., 2021) for evaluating code generation and synthesis capabilities of language models. This dataset comprises 974 hand-written programming tasks, ranging from basic to intermediate difficulty, designed to be suitable for introductory programming.



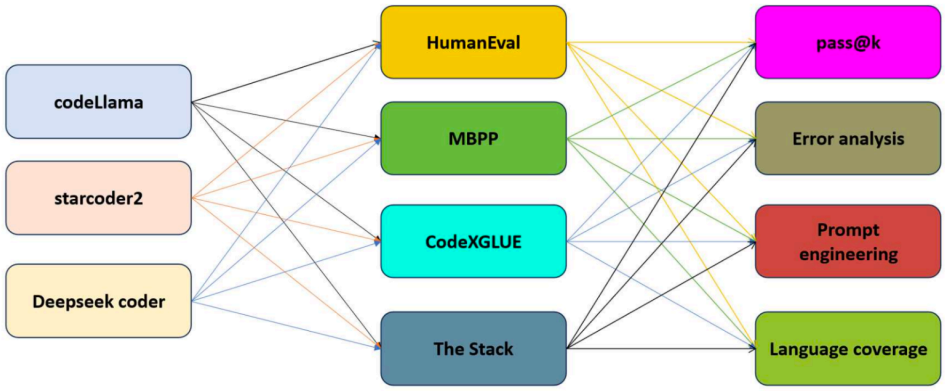




CodeXGLUE

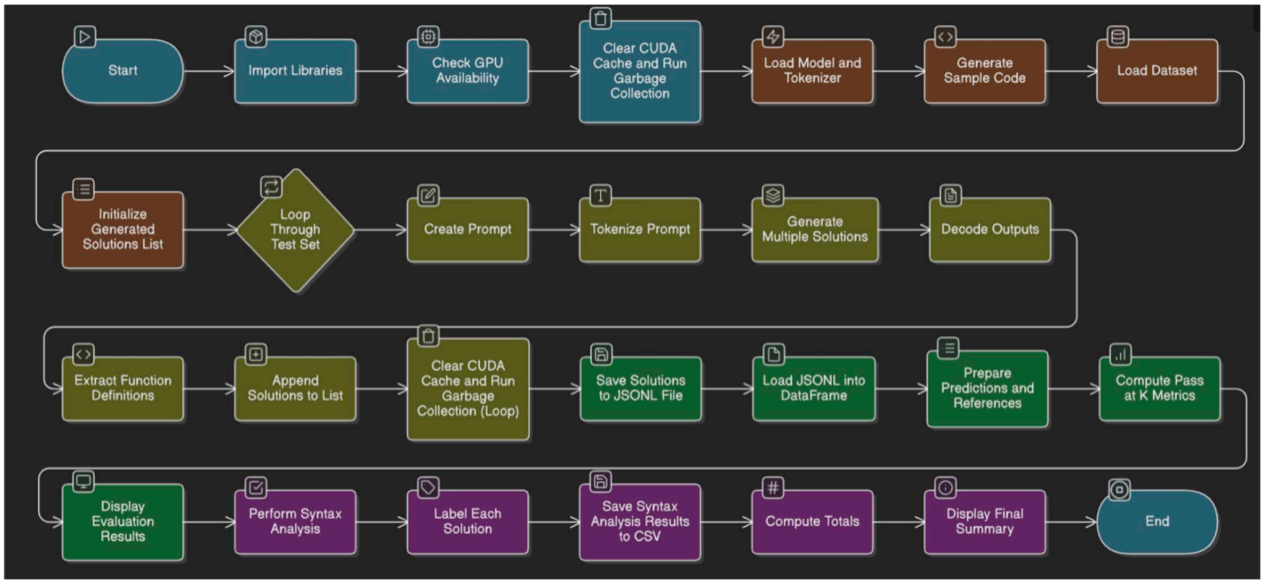
Code eXecution and Generation Language Understanding Evaluation Dataset (Introduced by Microsoft Research, 2020) to provide a standardized benchmark suite for training and evaluating models on code-related tasks like code generation, translation, classification, summarization, and more.

Evaluation Methodology: Flowchart



Implementation

End to End Flow Diagram



Implementation Details

We used following resources/artifacts:

- Open source pre-trained LLM models from HuggingFace
- Datasets (HumanEval/MBPP) from Huggingface
- Python evaluation package/scripts from Huggingface
- Google Colab environment with A100 GPU, 80GB memory configuration.

1. Install required packages

Install necessary packages for using HF transformers and the evaluation API.

```
%pip install transformers accelerate
%pip install evaluate datasets
```

2. Downloading the model

Use the transformers interface to download the pre-trained model checkpoints from HuggingFace.

```
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
checkpoint = "deepseek-ai/DeepSeek-Coder-V2-Lite-Base"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = AutoModelForCausalLM.from_pretrained(checkpoint,
dtype=torch.bfloat16).to(device)
```

3. Downloading the datasets

Download the HumanEval/MBPP datasets from Huggingface. (Please note that the same dataset can be cloned from github as well)

```
humaneval_dataset = load_dataset("openai_humaneval")
```

```
humaneval_dataset = load_dataset("mbpp", "sanitized")
```

4. Generating solutions

Generate solutions for each of the problem prompts from the dataset. Used pass@k metrics and generated up to 15 solutions using sampling.

```
generated_ids = model.generate(
    inputs["input_ids"],
    max_new_tokens=512,
    num_return_sequences=15, # Generate num_samples sequences
    pad_token_id=tokenizer.eos_token_id,
    attention_mask=inputs["attention_mask"],
    do_sample=True # Enable sampling to generate multiple sequences
)
```

5. Evaluate solutions

Download the evaluation package from huggingface and evaluate each of the solutions generated.

The CodeEval metric estimates the pass@k metric for code synthesis and calculates how good are predictions given a set of references.

[Please note that this metric exists to run untrusted model-generated code. Users are strongly encouraged not to do so outside of a robust security sandbox].

```
import evaluate
import os
# Set env to allow code evaluation & Load the evaluation metric
os.environ["HF_ALLOW_CODE_EVAL"] = "1"
code_eval_metric = evaluate.load("code_eval")
```

Generate pass@k results for k = 1, 5,10 and 15.

```
evaluation_results = code_eval_metric.compute(
   references=references,
   predictions=predictions,
   k=[1, 5, 10, 15] # Specify the k values for pass@k
)
```

6. AST analysis

An Abstract Syntax Tree (AST) is a tree-like data structure that represents the abstract syntactic structure of source code written in a formal language, such as a programming language.

7. LLM Temperature Study:

LLM temperature is a parameter that controls the randomness of a large language model's output, determining whether responses are more creative or more predictable. A low temperature (e.g. 0.2) makes the model more deterministic and focused, as it consistently picks the most likely next word, while a high temperature (e.g. 0.9) increases randomness, leading to more diverse and creative responses by giving less likely words a higher chance of being selected.

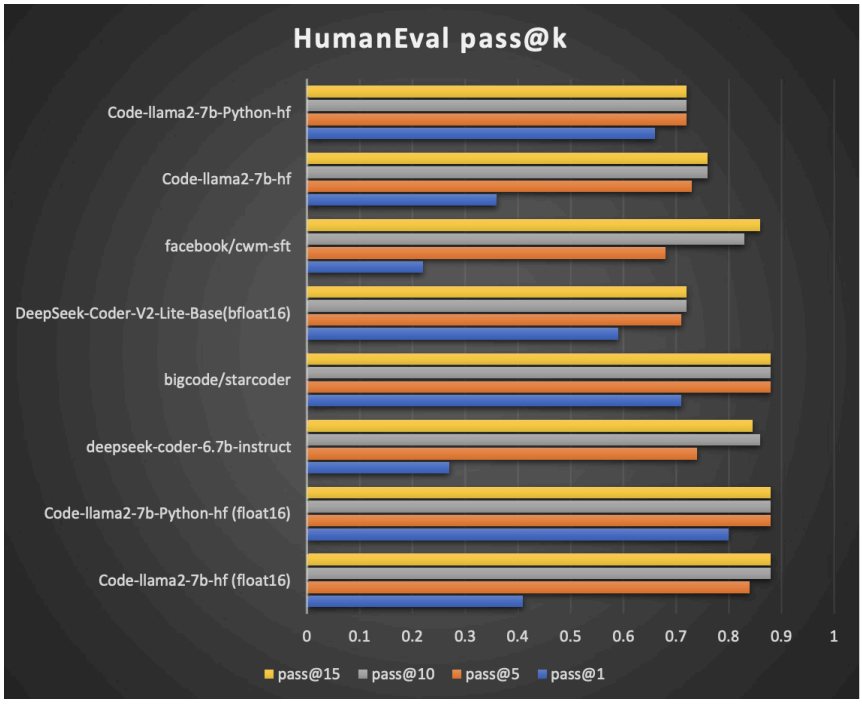
We studied effect temperature for one specific LLM i.e. deepseek-code-v2-Base

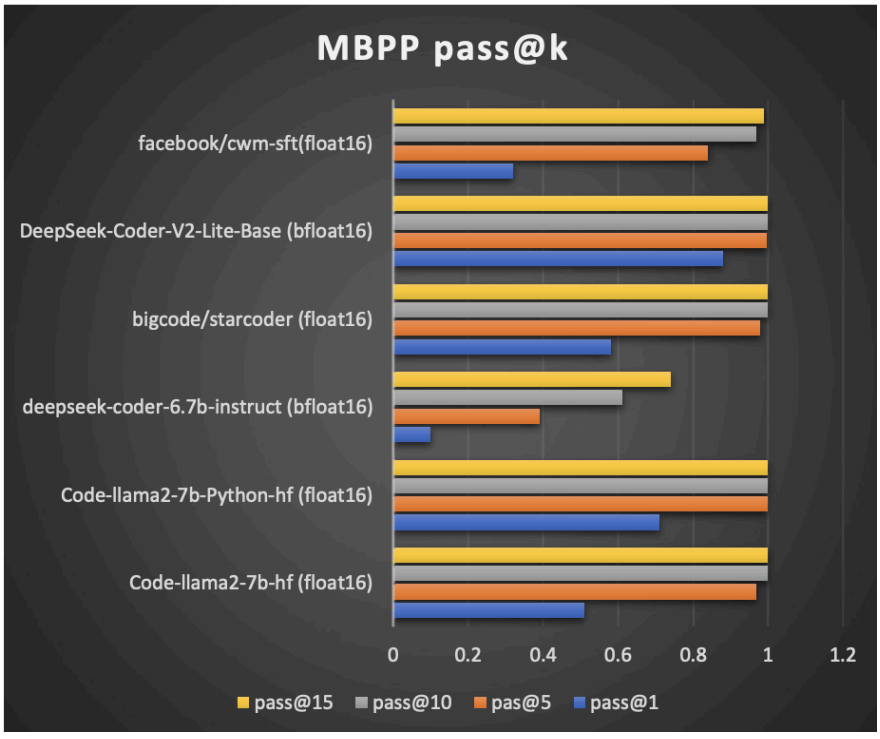
```
temperature_levels = [0.2, 0.5, 0.8, 1.0]
 generated_ids = model.generate(
           inputs["input_ids"],
           max_new_tokens=512,
           num_return_sequences=15,
           pad_token_id=tokenizer.eos_token_id,
           attention_mask=inputs["attention_mask"],
           temperature=temperature, # Add temperature
           do_sample=True if temperature > 0 else False,
 )
```

Evaluation Results

pass@k metrics

The charts below shows the results for Pass@k on both HumanEval and MBPP datasets





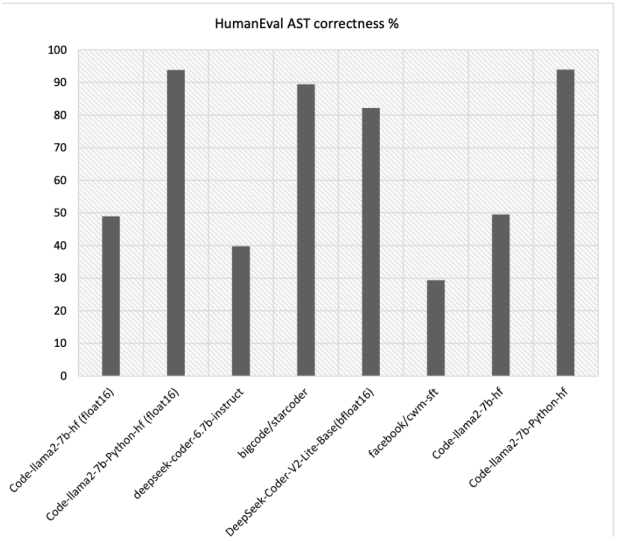
Key Findings and Inferences:

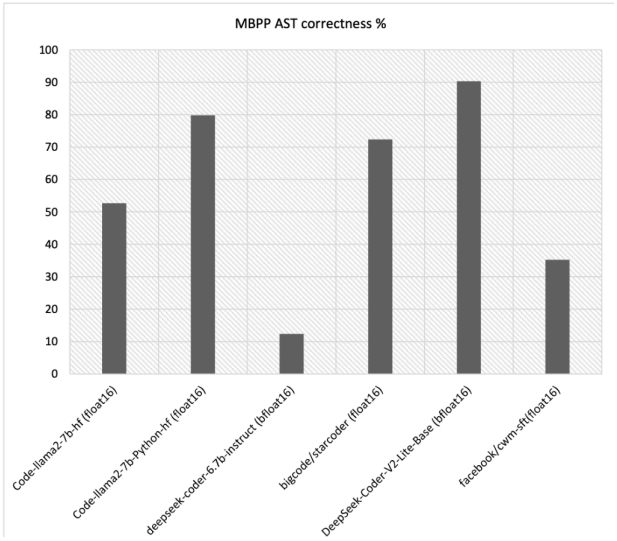
Specialization Advantage: Code-llama2-7b-Python-hf generally outperforms its base model counterpart (Code-llama2-7b-hf) across both HumanEval and MBPP benchmarks. This supports the conclusion that instruct-tuned and specialized variants often deliver superior results compared to using large, general foundation models without additional fine-tuning.

Model Performance Variation: While specific quantitative values are similar according to the chart bars (for example, the pass@1 results range from approximately 0.1 to 0.4 on HumanEval and it is only slightly higher on MBPP), the relative ranking shows significant differences.
The results show the comparative effectiveness of deepseek-coder-6.7b-instruct and bigcode/starcoder relative to the Code Llama variants.
Benchmark Consistency: The general trend of performance across models appears broadly consistent between both the Python-oriented benchmarks.

AST Correctness Comparison

The charts below show the percentage of syntactically accurate outputs for both HumanEval and MBPP.



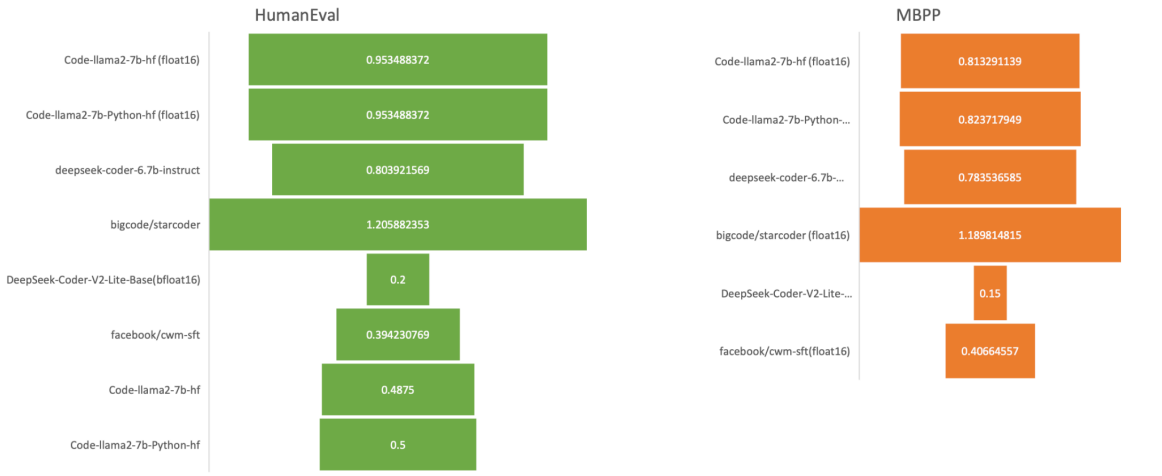


Key Findings and Inferences:

High Syntactic Accuracy: Code LLMs generally achieve high structural correctness percentages, often ranging near 80% to 90% or higher across the HumanEval and MBPP datasets. This confirms the prior research indicating that these models possess high syntax accuracy.
Disparity with Functional Results: While AST correctness is high, the pass@k results (functional correctness) are significantly lower. This large gap demonstrates the primary weakness: models struggle with logical reasoning required to satisfy unit tests, even when the generated code is syntactically perfect.

Model output Performance

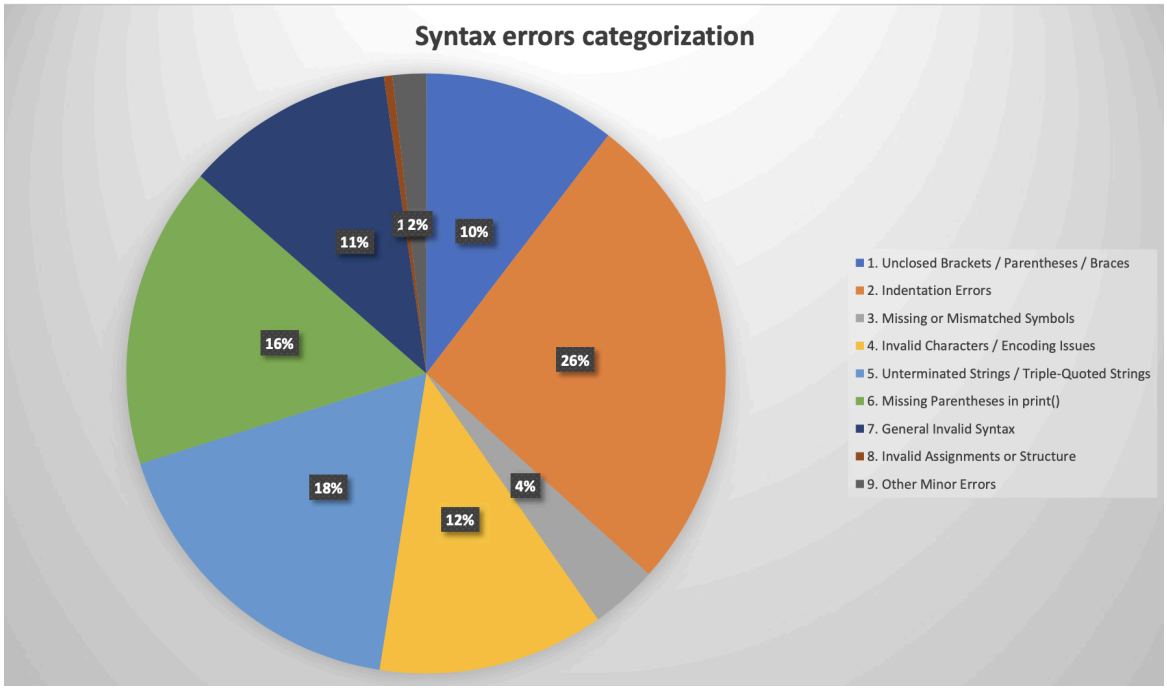
The throughput of models was measured in solutions per second (solutions / sec) for both HumanEval and MBPP



StarCoder's Efficiency: bigcode/starcoder exhibited the highest throughput, generating approximately 1.19 solutions/sec for MBPP and 1.21 solutions/sec for HumanEval.
Model Trade-off: The DeepSeek-Coder-V2-Lite model showed significantly lower throughput (around 0.15 to 0.2 solutions/sec). This illustrates a clear trade-off between the models - DeepSeek-V2-Lite leverages a large context window (128k), but its computational demand resulted in lower inference speed in this specific evaluation setup. Clear trade-offs exist between model size, compute cost, and specialization. Smaller models like CodeLlama-2-7B offer a balance of performance and lower GPU needs, making them ideal for academic or edge use.

Syntax Error Analysis

Analysed the type of syntax errors generated by the models for one specific LLM i.e. StarCoder with HumanEval dataset.

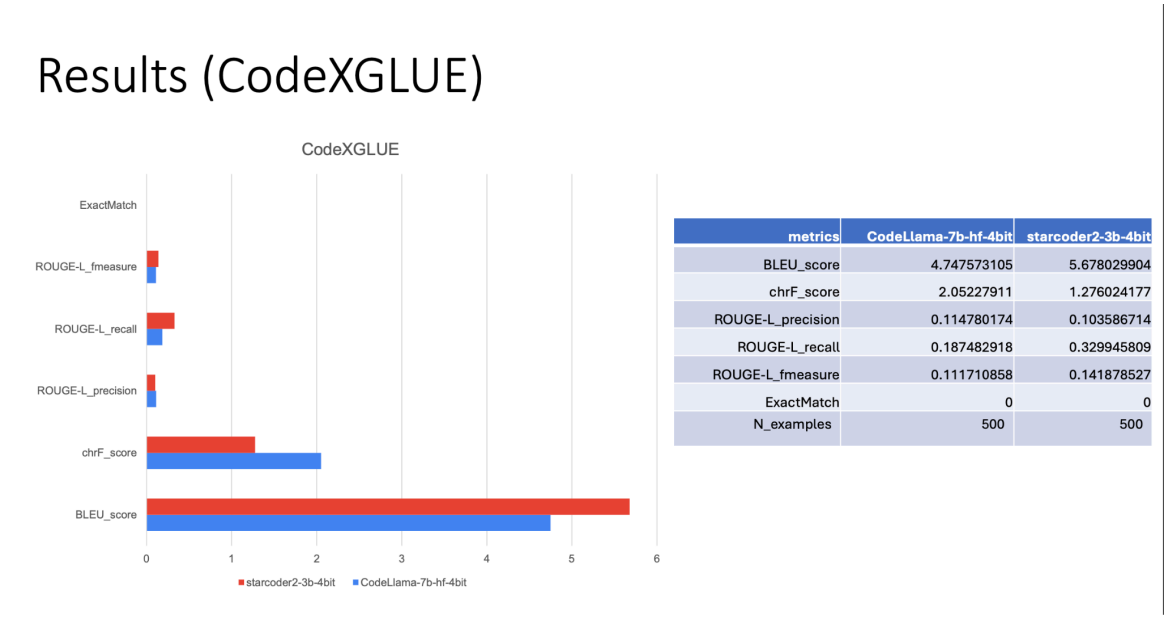


The most significantly frequent syntax errors that were observed include -

1. **Indentation Errors -** This represents the largest single category of syntax failure, indicating the model often struggles with Python's strict indentation requirements.
2. **Unterminated Strings -** This suggests difficulty managing complex string boundaries
3. **Missing Parentheses**

CodexGLUE Analysis

Evaluation on CodeXGLUE compared starcoder2-3b-4bit and CodeLlama-7b-hf-4bit using metrics related to code-to-text tasks.



metrics	CodeLlama-7b-hf-4bit	starcoder2-3b-4bit
BLEU_score	4.747573105	5.678029904
chrF_score	2.05227911	1.276024177
ROUGE-L_precision	0.114780174	0.103586714
ROUGE-L_recall	0.187482918	0.329945809
ROUGE-L_fmeasure	0.111710858	0.141878527
ExactMatch	0	0
N_examples	500	500

The results show that both the models achieved 0 exact matches out of the 500 examples, indicating that there is difficulty in generating perfectly matching outputs for the code to text tasks, which is to be expected.
It was also observed that there were chinese outputs that was generated for an English prompt during CodeXGLUE code-text tasks

Other Contexts & lesser known Languages

Other lesser known languages were also evaluated to benchmark their failures-

CUDA Code Generation: The CUDA-LLM "ByteDance-Seed/cudaLLM-8B" model(based on Qwen3-8B RL trained with verl for high performant CUDA kernel generation has been evaluated with “KernelBench”) consistently failed 100% to generate end-to-end functional code with a different dataset like “nvidia/compute-eval”. However, the model showed correct reasoning about how to implement the solution upon empirical analysis.

Complex Functional Failures: When prompted to create a complex task, such as a scientific calculator with detailed operations, all the models frequently failed to produce a complete and functionally correct solution

Errors for C-Code: When generating C programs, they often contained incorrect or missing function parameters and null pointer checks. Sometimes the generated code also contained Python syntax leading to compilation errors for the C program.

Context Misalignment: Sometimes the generated code was not aligned with the original prompt, generating completely unrelated solutions such as sorting algorithms for tasks that were unrelated to those.

Conclusions

The comparative analysis of Code LLMs (Code Llama, StarCoder, DeepSeek Coder) affirms that these models are fundamentally reshaping software workflows by enabling faster prototyping, code refactoring, and multi-language translation capabilities

- A clear hierarchy of trade-offs exists between model size, computational cost, and specialization. Smaller models (7B range) offer a compelling balance of performance and lower GPU demand
- The results confirmed that instruct-tuned and specialized variants, like Code Llama Python, provide demonstrable performance benefits over base models
- While models show high syntactic competency (high AST scores), their difficulty in overcoming logical flaws (lower pass@k scores) persists.
- Performance remains highly context-sensitive; models struggle particularly with complex, specialized domains (like CUDA) and cross-language generalization.

Recommendations for Future Work

Expanded Metrics: Incorporate more sophisticated evaluation metrics, including complexity score (O(n)), runtime performance, and inference time beyond simple throughput.
Real-World Evaluation: Move beyond standard benchmark datasets to evaluate models using actual real-world problems
Enhanced Training Methods: Explore the use of reinforcement learning (RL), integrating feedback loops derived from compilation and execution outcomes to improve model output reliability
Prompt Engineering Research: Investigate the accuracy impact of structured prompts versus natural language prompts
Security Analysis: Research the potential for code LLMs to generate malicious code
Advanced Frameworks: Study the application of Agentic AI and Retrieval-Augmented Generation (RAG) frameworks, including the usage of the Model Context Protocol (MCP)

References

1. StarCoder: may the source be with you

   [https://arxiv.org/pdf/2305.06161]

2. Code Llama: Open Foundation Models for Code

   [https://arxiv.org/pdf/2308.12950]

3. HumanEval: Evaluating Large Language Models Trained on Code

   [https://arxiv.org/pdf/2107.03374]

4. MBPP: Program Synthesis with Large Language Models

   [https://arxiv.org/pdf/2108.07732]

5. DeepSeekCoder: When the Large Language Model Meets Programming

   [https://arxiv.org/pdf/2401.14196]

6. CWM: An Open-Weights LLM for Research on Code Generation with

   World Models [https://arxiv.org/pdf/2510.02387]